

# QENLO: EMBEDDED AND OBSERVABLE FILTERED VECTOR SEARCH ACROSS CPUS AND COMMODITY GPUS

**Anonymous authors**

Paper under double-blind review

## ABSTRACT

Filtered vector search is often evaluated as an indexing problem and deployed as a service. That framing misses a smaller but consequential regime: applications that must keep data local, run inside a mobile or desktop process, and obtain predictable exact results across dissimilar accelerators. We present QENLO, an embedded Rust engine whose durable state is a canonical FP32 row store and whose CPU, GPU, and approximate indexes are rebuildable accelerators. Its exact GPU path evaluates predicates during a portable WGSF scan, computes bounded per-chunk top- $k$  lists, and merges them under a deterministic total order. The design exposes backend selection, fallback, transfers, and memory use rather than treating them as invisible runtime facts. The measurements do not support a universal speed claim. On one Windows workload with 100,000 384-dimensional vectors and all rows eligible, the shipped exact GPU path reduced median run P95 latency from 16.6567 to 3.2404 ms ( $5.14\times$ ; whole-run bootstrap 95% interval [3.55, 6.77]) at 0.99998 Recall@10. At 1% eligibility on the same host, exact CPU was instead  $12.5\times$  faster, motivating a selectivity-aware router. In a separate strict  $1M\times 768$  TopK Bench experiment on an RTX A6000, an unshipped CUDA prototype reached 0.170722 ms P95 and perfect recall, but remained  $1.29\times$  slower than FAISS GPU Flat. The result is therefore a systems case for portable embedded execution and observable routing, not a claim that QENLO dominates FAISS or approximate nearest-neighbor indexes.

## 1 INTRODUCTION

Vector retrieval systems commonly separate an application from its index behind a network boundary. The separation is useful at large scale, but it is not free: it creates another data copy, service, failure domain, and privacy boundary. A growing class of applications instead needs retrieval inside the process that owns the data: a desktop agent searching local notes, a mobile application retrieving private context, or a workstation tool that must continue without a network. In this setting, “fast search” is not a single kernel number. Package size, cold start, filter semantics, device availability, data durability, fallback behavior, and the latency of moving results back to the caller are part of the system.

Existing libraries cover important parts of this space. FAISS provides a broad family of exact and approximate CPU and CUDA indexes (Johnson et al., 2021; Douze et al., 2024). HNSW offers a strong graph-based latency–recall trade-off (Malkov & Yashunin, 2020); DiskANN and SPANN extend ANN search to SSD-backed billion-point collections (Subramanya et al., 2019; Chen et al., 2021). Filtered-ANN systems such as Filtered-DiskANN and ACORN integrate structured predicates into candidate generation (Gollapudi et al., 2023; Patel et al., 2024). These systems are not weak baselines, and several can be embedded on CPUs. The narrower gap addressed here is a first-class, observable, embedded execution path that can use non-CUDA GPUs through WebGPU-family APIs while retaining an exact CPU route for selective queries and unsupported devices.

This paper makes four contributions:

1. We describe an embedded architecture that separates canonical durable vectors from disposable accelerators and preserves deterministic filtered-search semantics across backends.

2. We formalize the correctness of chunk-local top- $k$  followed by a global merge, the operation that allows exact scans to fit finite GPU buffers.
3. We give a reproducible evaluation with retained failures and mismatched regimes kept separate. The evidence identifies a crossover: GPU scan wins for a dense 100k workload, while CPU scan wins decisively at 1% eligibility on the same machine.
4. We report a negative large-scale result. A CUDA research prototype is competitive with, but does not match, FAISS GPU Flat on the strict  $1\text{M} \times 768$  cell. This constrains the claim and exposes the remaining experiment needed for a native portable backend.

The work does not introduce a new ANN data structure. Its claim is that deployment constraints and observable routing are research-relevant system properties, and that honest crossover data is more useful than a single favorable headline.

## 2 PROBLEM AND SCOPE

Let  $X = \{(x_i, m_i, i)\}_{i=1}^n$  contain FP32 vectors  $x_i \in \mathbb{R}^d$ , metadata  $m_i$ , and stable identifiers  $i$ . Given a query  $q$ , predicate  $P(m_i)$ , and  $k$ , cosine search returns the first  $k$  eligible rows under the total order

$$\left(1 - \left\langle \frac{q}{\|q\|_2}, \frac{x_i}{\|x_i\|_2} \right\rangle, i\right). \quad (1)$$

The identifier breaks distance ties. A result is *exact* when every live eligible row is considered. Exactness does not imply bit-identical floating-point arithmetic between CPU FP64 oracles and GPU FP32 shaders; evaluation therefore checks identifiers against an independent oracle and reports  $\text{Recall}@k$ .

The target is an in-process collection small enough to reside on one device. This excludes distributed sharding and service-level replication. It includes durability, because an embedded index that loses acknowledged writes is not useful, and it includes fallback, because commodity devices do not offer a uniform accelerator surface.

## 3 SYSTEM DESIGN

### 3.1 CANONICAL STATE AND DISPOSABLE ACCELERATION

QENLOstores normalized FP32 vectors, metadata, tombstones, and stable row identifiers as its canonical state. Exact CPU and GPU views, IVF-Flat, IVF-SQ8, and optional HNSW acceleration are derived state. A crash-safe snapshot is written to a temporary file, synchronized, and atomically renamed; the previous valid snapshot remains readable until replacement. Checksums and versioned headers reject truncated or incompatible files.

This separation has two practical consequences. First, changing a search backend does not change the durable representation. Second, a missing GPU or corrupt accelerator can cause a visible rebuild or fallback rather than data loss. Approximate indexes produce candidates that are reranked against canonical FP32 rows; they are approximate only because unprobed rows can be absent.

### 3.2 PREDICATE EXECUTION AND EXACT GPU SCAN

The host compiles supported metadata predicates into compact GPU buffers. Depending on the predicate and eligible count, execution uses one of three forms: evaluate the predicate in the shader, materialize eligible row identifiers, or construct a host mask. The exact GPU kernel computes cosine distance for every eligible row. Work is divided into chunks to bound storage and readback. Each chunk emits no more than  $k$  candidates, after which the host performs one final merge.

**Theorem 1** (Exact chunk merge). *Let  $C_1, \dots, C_m$  partition the eligible rows under a deterministic total order  $\prec$ . Let  $T_j$  be the first  $k$  items of  $C_j$ . The global first  $k$  items of  $\bigcup_j C_j$  are contained in  $\bigcup_j T_j$ .*

*Proof.* If  $z \in C_j$  but  $z \notin T_j$ , then  $C_j$  contains at least  $k$  distinct items preceding  $z$ . Those items also precede  $z$  globally, so  $z$  cannot be in the global first  $k$ . Consequently, discarding items outside each  $T_j$  cannot discard a global top- $k$  item. Merging the retained candidates under the same order returns the exact result.  $\square$

The theorem is a correctness argument, not an algorithmic novelty claim. Its engineering value is that output traffic scales with chunks times  $k$ , rather than with the number of scanned rows.

### 3.3 PORTABLE DEVICE PATH

The shipped GPU backend is implemented with `wgpu` and WGSL, following the WebGPU execution and shader model (W3C GPU for the Web Working Group, 2026a;b). On Windows it can select DirectX 12 or Vulkan; Vulkan also provides a route to Linux and Android, while Metal is the corresponding Apple backend. This is a deployment distinction from CUDA-only GPU indexes, not a statement that FAISS or USearch cannot run in an embedded CPU process. Physical Android and iOS performance remains unmeasured in the current study.

### 3.4 OBSERVABLE ROUTING

Backend choice is reported with the result diagnostics: requested backend, executed backend, fallback reason, filter strategy, candidate counts, transfer bytes, owned GPU allocations, and timing phase. The automatic route treats the eligible-row count and batch size as first-class inputs. This follows directly from Section 5: dispatch overhead dominates sparse single-query work, while a dense scan amortizes it. A route decision is therefore a measured crossover, not a device-brand heuristic.

On x86-64, the exact CPU dot product dispatches at runtime among scalar, SSE2, AVX2, and AVX2+FMA implementations while retaining FP64 accumulators. During this work we widened the AVX2 loop to two independent eight-element accumulation lanes and added an AVX2+FMA path. A proposed allocation-removal rewrite was rejected after a controlled development run roughly doubled P95; it is not present in the final implementation. We do not claim an end-to-end SIMD speedup because the available before/after runs were sequential and thermally unstable.

## 4 EXPERIMENTAL METHOD

### 4.1 RESEARCH QUESTIONS

We ask: (RQ1) when does portable exact GPU execution improve in-process latency over exact CPU execution? (RQ2) how does QENLcompare with established libraries at matched recall? (RQ3) does the available  $1\text{M} \times 768$  result establish a performance advantage over FAISS? (RQ4) which results transfer to embedded-device deployment?

### 4.2 METRICS AND TIMING BOUNDARY

The primary metric is the median, across repetitions, of per-run P95 latency. Recall@10 is computed against an exhaustive oracle. The in-process boundary starts at the search call and ends when ordered identifiers and distances are available to the host. It includes predicate work, host-device transfer, synchronization, readback, and merge; it excludes loading, index build, and oracle construction. Build time is reported separately when available. A result is eligible for a comparison only when workload, hardware, query order, filter semantics, and API boundary match.

For the five-run Windows comparison we use a seeded, independent whole-run bootstrap with 10,000 draws and report the percentile interval of the ratio of medians. Five repetitions yield a coarse interval; it captures run-to-run sampling, not changes in hardware, thermals, or workload.

### 4.3 DATASETS AND SYSTEMS

The  $100\text{k} \times 384$  Windows cohort uses locally generated AG News sentence embeddings derived from the public DTU artifact (Nielsen, 2022), with disjoint corpus, tuning, and evaluation ranges and

Table 1: Matched Windows 100k×384 exact-search results. Lower P95 is better.

| Eligibility | Backend             | P95 (ms)      | Recall@10 | Runs |
|-------------|---------------------|---------------|-----------|------|
| 100%        | Qenlo CPU           | 16.6567       | 0.99998   | 5    |
| 100%        | Qenlo GPU predicate | <b>3.2404</b> | 0.99998   | 5    |
| 1%          | Qenlo CPU           | <b>0.2304</b> | 1.00000   | 5    |
| 1%          | Qenlo GPU predicate | 2.8832        | 1.00000   | 5    |

Table 2: Strict TopK Bench 1M×768, 1% filter, RTX A6000. Each valid row has 5,000 samples.

| System               | P50 (ms)        | P95 (ms)        | P99 (ms)        | Recall@10 |
|----------------------|-----------------|-----------------|-----------------|-----------|
| NumPy CPU exact FP32 | 0.121747        | 0.197311        | 0.228314        | 1.0000    |
| FAISS GPU Flat IP    | <b>0.126051</b> | <b>0.132222</b> | <b>0.145826</b> | 1.0000    |
| Qenlo CUDA prototype | 0.160011        | 0.170722        | 0.180932        | 1.0000    |
| cuVS brute force     | —               | —               | —               | —         |

synthetic metadata independent of vector content. It uses 200 warmups, 5,000 evaluation queries, five repetitions, batch 1, and  $k = 10$ .

The strict large cell uses TopK Bench at commit 398ce7d72dea7ad765ada54c5daa014c498efb52 (TopK, 2026): one million 768-dimensional document vectors, 1,000 held-out queries, predicate `int_filter < 100`, 100 warmups, five repetitions, batch 1, and  $k = 10$ . Temporary FP32 copies are normalized and ranked by inner product; an independent CPU FP64 exhaustive oracle operates on those working vectors. The GPU is an RTX A6000. FAISS uses `GpuIndexFlatIP`. The QENLOcell is a PyTorch CUDA predicate prototype, isolated from FAISS because the library preload needed by FAISS caused a PyTorch GEMV failure. It is not the shipped Rust/WGSL backend.

A retained Linux 100k×384 cohort compares Qenlo CPU, USearch, FAISS, and Chroma. It has only three repetitions and a different host and corpus checksum, so it is descriptive and is never pooled with the Windows cohort. Its GPU cell failed adapter creation and remains recorded as a failure.

## 5 RESULTS

### 5.1 DENSE AND SELECTIVE EXACT SEARCH ON ONE HOST

For all-row search, GPU P95 is  $5.1403\times$  lower than CPU P95, with a whole-run bootstrap 95% interval of [3.5521, 6.7749]. For 1% eligibility, the direction reverses: CPU is  $12.51\times$  faster. This answers RQ1 only for this host and these two cells. It also falsifies a simple “GPU whenever available” policy. The practical optimization is routing: compact sparse work on the CPU and reserve device dispatch for enough arithmetic to amortize fixed costs.

An Intel Arc device-lab soak provides a second, non-NVIDIA observation at 100k×384: exact GPU P95 was 4.444 ms versus exact CPU P95 16.486 ms, with Recall@10 1.0 over 512 samples. IVF-Flat reached 2.704 ms. This is evidence that the shader path can execute through Vulkan; one device is not a cross-platform performance matrix.

### 5.2 STRICT ONE-MILLION-VECTOR GATE

The Qenlo prototype is  $1.29\times$  slower than FAISS at P95 and only  $1.16\times$  lower than the retained CPU reference. It fails the preregistered requirement of at least  $2\times$  lower P95 than the fastest recall-qualified baseline. cuVS produced no valid sample: after correcting an adapter return-order error, runs failed on missing or incompatible runtime libraries. The raw failures are retained. Thus RQ3 has a direct answer: no performance advantage is demonstrated at the strict gate.

The result is still informative. The compact predicate-specialized prototype is within 29% of a mature FAISS flat GPU implementation while returning exact identifiers, but “within 29%” is not “on par” under a claim of statistical equivalence. Equivalence would require a prespecified margin and paired repetitions of the shipped implementation.

Table 3: Retained Linux 100k×384 all-row cohort. Different host and three runs; descriptive only.

| System      | Search type | P95 (ms)      | Recall@10 |
|-------------|-------------|---------------|-----------|
| Qenlo CPU   | exact       | 12.9014       | 1.0000    |
| USearch     | approximate | 5.0933        | 0.9914    |
| FAISS Flat  | exact       | 10.2171       | 1.0000    |
| FAISS HNSW  | approximate | <b>1.4939</b> | 0.9994    |
| Chroma HNSW | approximate | 3.7767        | 0.9984    |

### 5.3 DESCRIPTIVE LIBRARY COMPARISON

Qenlo CPU does not lead this cohort. FAISS Flat is  $1.26\times$  faster at exact search; FAISS HNSW is  $8.64\times$  faster at slightly lower recall. This answers RQ2 narrowly and reinforces the paper’s scope: embedded portability and explicit runtime behavior, not a new universally superior index. Build and persistence boundaries differ among Chroma and in-process libraries; treating the table as a product leaderboard would be invalid.

## 6 WHAT THE BENCHMARKS CHANGED

The experiments led to code and policy changes rather than post-hoc narrative changes alone. First, the automatic route gives sparse single-query filters a CPU path because the 1% crossover is adverse. Second, CPU dispatch now distinguishes AVX2+FMA and uses independent FP64 accumulation lanes to reduce dependency length. Third, raw-corpus import verifies the checksum of the complete source while allowing a reproducible leading-row subset, preventing a benchmark convenience from weakening provenance. Fourth, failed backends and rejected optimizations remain artifacts: the allocation-removal experiment was reverted, cuVS failures remain visible, and the CUDA prototype is labeled unshipped wherever it appears.

These changes embody a simple systems principle: an optimization is a committed result only when its semantics, provenance, and relevant latency distribution survive measurement. Source-level plausibility is insufficient.

## 7 RELATED WORK

FAISS combines exact flat search, quantization, inverted files, graphs, and optimized GPU primitives (Johnson et al., 2021; Douze et al., 2024). Qenlo uses FAISS as the strongest exact GPU baseline available in the strict cohort. HNSW obtains high-recall ANN through a navigable hierarchical proximity graph (Malkov & Yashunin, 2020); USearch and Chroma expose related graph-based trade-offs in the retained small cohort.

DiskANN stores a graph index across memory and SSD, and SPANN uses an in-memory centroid structure with disk posting lists (Subramanya et al., 2019; Chen et al., 2021). Their target is collections larger than device memory. Qenlo instead targets a single embedded process and deliberately keeps canonical vectors available for exact reranking and rebuilds.

Filtered-DiskANN studies label-constrained ANN at web scale (Gollapudi et al., 2023). ACORN extends HNSW with predicate-aware traversal for general structured predicates (Patel et al., 2024). These works change how approximate candidates are reached. Qenlo’s exact predicate scan makes a different trade: linear eligible work and exact coverage in exchange for a simple rebuildable accelerator and predictable semantics. Its optional IVF and HNSW routes should be understood as complements, not evidence that exhaustive scanning replaces ANN at scale.

WebGPU standardizes a portable host API and WGSL shader language (W3C GPU for the Web Working Group, 2026a;b). Portability at the API level does not guarantee equal device behavior: adapter limits, timestamp support, driver quality, unified versus discrete memory, and shader compilation vary. The device-lab result establishes execution on one Vulkan GPU, not performance on Android or Metal.

## 8 LIMITATIONS AND THREATS TO VALIDITY

The central limitation is that the strongest large-scale number is not from the shipped backend. The native Rust/WGSL path has not completed the strict  $1\text{M} \times 768$  gate. The CUDA prototype and FAISS ran in isolated processes because of a runtime-library conflict. Although corpus, query order, hardware, oracle, and timing boundary match, process isolation is a residual threat.

The evidence covers one Windows laptop GPU, one Intel Arc device-lab run, and one A6000 prototype run. There are no physical Android, iOS, or Apple Silicon latency, energy, thermal-throttling, or package-size measurements. Consequently, RQ4 is answered only at the level of software architecture and one non-NVIDIA execution result. “Runs on device” must not be read as “has been benchmarked on every target device.”

Metadata in the 100k workload is synthetic and independent of vector geometry. Correlated and adversarial filters can change ANN recall and traversal cost. The study lacks multi-seed corpora, concurrency sweeps, 10M vectors, build-memory peaks, energy, and matched service-boundary systems. The Windows ratio uses five runs, so its bootstrap interval is intentionally described as coarse.

Finally, the paper evaluates a system under active development. The SIMD change is tested for correctness but does not carry a performance claim; accepting a sequential before/after number would confound code with machine state. A publishable final evaluation should freeze a commit, pre-register the matrix, randomize adapter order within feasible constraints, and rerun from a clean environment.

## 9 CONCLUSION

Qenlo shows that filtered vector search can be built as an embedded, durable, observable component with an exact GPU route that is not tied to CUDA. The favorable result is specific but substantial:  $5.14 \times$  lower P95 than exact CPU for one dense  $100\text{k} \times 384$  workload. The unfavorable results are equally central: CPU wins by  $12.5 \times$  at 1% eligibility on the same host, FAISS GPU Flat leads the strict A6000 cell by  $1.29 \times$ , and the large-scale Qenlo result is still a prototype. These facts do not support a universal speed claim. They support a narrower design: measure selectivity, route explicitly, retain canonical state, expose fallback, and make portable acceleration an implementation choice rather than a storage contract.

### REPRODUCIBILITY STATEMENT

Source, benchmark runners, raw per-query samples, checksums, environment captures, failed runs, and statistical summaries are included in the accompanying repository. The paper source records the author identity, while the compiled review manuscript remains anonymous under the official ICLR 2027 style. Commands and artifact locations are listed in `paper/README.md`.

## REFERENCES

- Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. SPANN: Highly-efficient billion-scale approximate nearest neighborhood search. In *Advances in Neural Information Processing Systems*, volume 34, pp. 5199–5212, 2021. URL [https://proceedings.neurips.cc/paper\\_files/paper/2021/hash/299dc35e747eb77177d9cea10a802da2-Abstract.html](https://proceedings.neurips.cc/paper_files/paper/2021/hash/299dc35e747eb77177d9cea10a802da2-Abstract.html).
- Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvassy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The Faiss library. *arXiv preprint arXiv:2401.08281*, 2024. URL <https://arxiv.org/abs/2401.08281>.
- Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, and Yin Zhang. Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters. In *Proceedings of the ACM Web Conference 2023*, pp. 3406–3416, 2023. doi: 10.1145/3543507.3583552.
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021. doi: 10.1109/TBDATA.2019.2921572.

Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. doi: 10.1109/TPAMI.2018.2889473.

Beatrix Miranda Ginn Nielsen. DTU pretrained sentence BERT AG News embeddings, 2022.

Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. ACORN: Performant and predicate-agnostic search over vector embeddings and structured data. *Proceedings of the ACM on Management of Data*, 2(3), 2024. doi: 10.1145/3654923.

Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Abstract.html>.

TopK. TopK Bench: Vector database benchmarks. <https://github.com/topk-io/bench>, 2026. Commit 398ce7d72dea7ad765ada54c5daa014c498efb52; accessed 2026-09-02.

W3C GPU for the Web Working Group. WebGPU specification. <https://www.w3.org/TR/webgpu/>, 2026a. Accessed 2026-09-02.

W3C GPU for the Web Working Group. WebGPU Shading Language. <https://www.w3.org/TR/WGSL/>, 2026b. Accessed 2026-09-02.

## A ARTIFACT MAP

The matched 100k comparison is stored under `benchmarks/2026-08-28/real/comparisons/`. The retained cloud audit is under `benchmark-results/2026-09-02-runpod-archive/`. The strict A6000 artifacts, including cuVS failures and independent oracle output, are under `benchmark-results/2026-09-02-runpod-a6000-strict-research-gate/`. The provider-compatible TopK collection is retained separately and excluded from the strict headline because its inclusive predicate produced 10,148 rather than the requested strict eligible set.

## B CLAIM LEDGER

Table 4: Claims and the evidence required to use them.

| Claim                                   | Current status                           | Required next evidence                          |
|-----------------------------------------|------------------------------------------|-------------------------------------------------|
| Dense 100k GPU lower P95 than Qenlo CPU | Supported on one Windows host            | Replication on frozen commit and more devices   |
| Sparse filters should route to CPU      | Supported for one 1% cell                | Selectivity sweep with randomized run order     |
| Competitive with FAISS GPU at 1M        | Descriptive only; 1.29× slower prototype | Native backend, equivalence margin, paired runs |
| Portable on-device performance          | Not established                          | Physical Android, iOS, Apple Silicon matrix     |
| Universally fastest filtered search     | Falsified by present cohorts             | No such claim should be made                    |